```
import tkinter as tk
import turtle
from PIL import ImageTk,Image
from tkinter import ttk
from tkinter import *

# Unchanged code
o = 105
Max = 250
n = 0
gra = [[0 for _ in range(Max)] for _ in range(Max)]
dx = [1, 0, 0, -1]
dy = [0, 1, -1, 0]
vis = [[False for _ in range(Max)] for _ in range(Max)]
Direction = ['e', 'n', 's', 'w']
sta = [0] * 30
num = [0, 3]
blocked_coordinates = []
blocked_count = 0


def dfs(x, y, pos, dir, golygon_count, golygon_seq):
    global n
    if pos == n + 1:
        if x == o and y == o:
            golygon_seq[0] = ''.join([Direction[sta[i]] for i in range(1, n + 1)])
            golygons_text.insert(tk.END, golygon_seq[0] + "\n")
            golygon_count[0] += 1
        return

    step = sum(range(pos, n + 1))
    if abs(x - o) + abs(y - o) > step:
        return

    if dir == -1:
        for i in range(4):
            xx, yy = x + pos * dx[i], y + pos * dy[i]
            if vis[xx][yy]:
                continue
            if 1 <= i < 3:
                for k in range(min(y, yy), max(y, yy) + 1):
                    if gra[xx][k]:
                        break
                else:
                    sta[pos] = i
                    vis[xx][yy] = True
                    dfs(xx, yy, pos + 1, 1, golygon_count, golygon_seq)
                    vis[xx][yy] = False
            else:
                for k in range(min(x, xx), max(x, xx) + 1):
                    if gra[k][yy]:
                        break
                else:
                    sta[pos] = i
                    vis[xx][yy] = True
                    dfs(xx, yy, pos + 1, 0, golygon_count, golygon_seq)
                    vis[xx][yy] = False
    else:
        if dir == 0:
            for i in range(1, 3):
                xx, yy = x + pos * dx[i], y + pos * dy[i]
                if vis[xx][yy]:
                    continue
                for k in range(min(y, yy), max(y, yy) + 1):
                    if gra[xx][k]:
                        break
                else:
                    sta[pos] = i
                    vis[xx][yy] = True
                    dfs(xx, yy, pos + 1, 1, golygon_count, golygon_seq)
                    vis[xx][yy] = False
        elif dir == 1:
            for j in range(2):
```

```
                i = num[j]
                xx, yy = x + pos * dx[i], y + pos * dy[i]
                if vis[xx][yy]:
                    continue
                for k in range(min(x, xx), max(x, xx) + 1):
                    if gra[k][yy]:
                        break
                else:
                    sta[pos] = i
                    vis[xx][yy] = True
                    dfs(xx, yy, pos + 1, 0, golygon_count, golygon_seq)
                    vis[xx][yy] = False


def draw_golygon(golygon_seq):
    x, y = 0, 0
    dis = 100
    turtle.penup()
    turtle.goto(x, y)
    turtle.pendown()

    turtle.setheading(0)
    for direction in golygon_seq:
        if direction == 'n':
            turtle.setheading(90)
            turtle.forward(dis) # Increased size

        elif direction == 'e':
            turtle.setheading(0)
            turtle.forward(dis) # Increased size

           # draw_direction("E")
        elif direction == 's':
            turtle.setheading(270) # Change heading for a 90-degree turn
            turtle.forward(dis) # Increased size

           # draw_direction("S")
        elif direction == 'w':
            turtle.setheading(180)
            turtle.forward(dis) # Increased size
            #draw_direction("W")
        dis += 20
    turtle.penup()


'''def draw_direction(direction):
    x, y = turtle.pos()
    if direction == "N":
        x -= 10
        y += 30
    elif direction == "E":
        x += 20
        y -= 10
    elif direction == "S":
        y -= 40
    elif direction == "W":
        x -= 40
    turtle.goto(x, y)
    turtle.write(direction, align="center")'''


# GUI functions
def add_blocked_coordinate():
    global blocked_count
    if blocked_count < int(blocked_entry.get()):
        entry = tk.Entry(blocked_coordinates_frame)
        entry.pack(pady=5)
        blocked_coordinates.append(entry)
        blocked_count += 1


def solve_golygons():
    global n, gra
```

```
    n = int(length_entry.get())
    blocked_count = int(blocked_entry.get())
    gra = [[0 for _ in range(Max)] for _ in range(Max)]
    for i in range(blocked_count):
        x, y = map(int, blocked_coordinates[i].get().split())
        if abs(x) <= o and abs(y) <= o:
            gra[o + x][o + y] = 1

    golygon_count = [0] # Use a list to store the count
    golygon_seq = [''] # Initialize as list to mimic the original code logic
    dfs(o, o, 1, -1, golygon_count, golygon_seq)
    if golygon_count[0] > 0:
        golygons_text.insert(tk.END, "Found {} golygon(s).\n".format(golygon_count))
        turtle.hideturtle()
        turtle.penup()
        turtle.goto(0, 0)
        turtle.setheading(0)
        draw_golygon(golygon_seq[0])
        turtle.update()
        # golygons_text.insert(tk.END, "Found 1 golygon(s).\n")
    else:
        golygons_text.insert(tk.END, "Found 0 golygon(s).\n")

    turtle_screen.update()
    turtle_screen.mainloop()



import tkinter as tk
from tkinter import PhotoImage
from PIL import Image

# Open the JPEG image
jpeg_image = Image.open("C:\Users\harsh\OneDrive\Desktop\image.jpg") # Replace with your image file

# Convert and save as GIF
gif_image_path = "image.gif"
jpeg_image.save(gif_image_path, format="GIF")
#root = tk.Tk()
#root.title("Background Image Example")
#root.geometry("800x600")

root = tk.Tk()
root.title("Canvas with Background Image")

canvas = tk.Canvas(root, width=1600, height=1100)
canvas.pack()

# Load and display the background image on the canvas
background_image = Image.open("image.gif")
background_image_resize = background_image.resize((1600, 1100), Image.LANCZOS)

background_photo = ImageTk.PhotoImage(background_image_resize)
canvas.create_image(0, 0, anchor=tk.NW, image=background_photo)

# Add other elements on the canvas
#label = tk.Label(canvas)
#label.place(x=700, y=120)
length_label = tk.Label(canvas, text="Length of Longest Edge:", font=("Arial", 18))
length_label.place(x=300,y=160)

# Create a larger entry widget
length_entry = tk.Entry(root, width=30) # Adjust the width as needed
length_entry.place(x=700, y=160) # Set the coo
blocked_label = tk.Label(root, text="Number of Blocked Intersections:", bg="orange",font=("Arial", 18))
blocked_label.place(x=300,y = 220)
blocked_entry = tk.Entry(root,width=30)
blocked_entry.place(x=700,y=220)

blocked_coordinates_frame = tk.Frame(root, bg="orange")
blocked_coordinates_frame.place(x=300,y=320)

   add_blocked_button = tk.Button(root, text="Add Blocked Intersection", command=add_blocked_coordinate, bg="blue",
```

```
                                        fg="orange",font=("Arial", 18))
add_blocked_button.place(x=300,y=380)

solve_button = tk.Button(root, text="Solve", command=solve_golygons, bg='green')
```